

CalmGuide

System Architecture Overview

A multilingual AI companion for dementia caregivers — Moment Coach, Daily Check-In, behavioral pattern tracking, and a B2B facility portal, built on a safety-gated LLM pipeline with retrieval-augmented grounding.

Prepared: September 2026 | Repo state: commit 12151f4 (main)

Web: Next.js 16 / React 19 · Mobile: Expo SDK 55 · Backend: FastAPI / Python 3.12 · DB: PostgreSQL 16 + pgvector

1 Product Overview

CalmGuide is a multilingual (English, Spanish, Hindi) AI companion for dementia caregivers — family members, typically aged 55–70, managing a loved one with dementia at home. There is no username/password account: a caregiver signs in with a private, one-way-hashed 8-character **access code**, generated on profile creation and gated behind an invite code in production.

Core tools

- **Moment Coach** — describe what's happening right now (typed or spoken); get a structured four-part response: *Right Now* (60-second actions), *Why This Is Happening*, *When to Get More Help*, and *What NOT to Do*.
- **Daily Check-In** — a lighter, reflective prompt about the caregiver's own wellbeing, safety-screened the same way as Moment Coach.
- **Learn Mode** — scenario practice with structured AI feedback.
- **Behavioral pattern tracking** — incident auto-extraction, episode-cycle risk prediction ("Tonight's Outlook"), and cross-patient anonymized strategy recommendations.
- **Facility Portal (B2B)** — staff accounts, resident assignment, incident tracking, audit log, admin/executive dashboards, for care facilities.

Every message is run through a two-layer safety pipeline **before** it reaches the LLM. A message flagged as a crisis is routed straight to localized emergency resources (911/988/Alzheimer's Helpline, or the correct regional equivalent) instead of an AI-generated answer. Responses are grounded in a caregiver-built care profile plus a retrieval-augmented (RAG) layer over real caregiving reference material — the Alzheimer's Association, Mayo Clinic, HelpGuide, Family Caregiver Alliance, and CDC (41 pages / 231 chunks).

■ Explicitly non-clinical

The product's own documentation (docs/SAFETY_ARCHITECTURE.md, services/dice_workflow.py) states plainly that the safety pipeline has not received clinical validation and carries TODO(CLINICAL-REVIEW-REQUIRED)/TODO(CLINICAL-LEGAL-REVIEW) markers on unresolved areas (e.g. no mandatory-reporting / duty-to-warn logic exists yet for elder-abuse or caregiver-harm-risk disclosures).

2 High-Level Architecture

```
Browser (Next.js 16, React 19, TS) ■■SSE / REST■■■■ Backend (FastAPI, async SQLAlchemy)
■
Mobile (Expo SDK 55, expo-router) ■■same REST / SSE■■■■ ■■■■ OpenAI / Anthropic (chat
completions)
■■■■ OpenAI (embeddings, neural TTS)
▼
PostgreSQL 16 + pgvector
(profiles · conversations · rag_chunks
incidents · dossiers · facilities · audit_log)
```

Both clients speak the same REST/SSE contract to a single backend — there is no client-specific API surface.

Repository layout

Path	Stack	Deploys to
frontend/	Next.js 16 App Router, TypeScript (strict), Tailwind, next-intl	Vercel
mobile/	Expo SDK 55, expo-router, react-i18next	Sideloaded Android APK + EAS OTA
backend/	FastAPI, async SQLAlchemy 2.0, Pydantic v2, Alembic	Railway (Docker)
rag/	Standalone scrape → chunk → embed → pgvector pipeline (own venv)	Runs against the same Postgres
locales/	Source-of-truth translation JSON, synced into both clients	N/A (build-time)
docs/	Safety architecture, audit log, milestone reports, deferred-features log	N/A

Infra migration: the app originally ran on a collaborator-hosted backend outside the team's control. It was migrated to fully self-owned infra — Vercel (frontend) and Railway + Docker (backend + Postgres) — after evaluating and rejecting Oracle Cloud's free tier (blocked by capacity exhaustion across all availability domains) and considering Fly.io / Cloudflare Workers.

3 Backend Architecture

Structure

Directory	Contents
routers/	One file per API surface: coach, checkin, checkin_daily, profile, feedback, insights, prediction, learn, incidents, care_changes, speech, languages, impact, health, plus facility_* (B2B).
services/	LLM abstraction (llm_provider, openai_provider, anthropic_provider, fallback_provider); safety (safety_gate, safety_classifier, safety_log, safety_redteam, response_guard); crypto/auth (crypto, auth, jwt_service, facility_auth); RAG glue (retrieval_query); memory graph (dossier, incident_extractor/retriever, pattern_detector, cross_patient, insights); plus rate_limit, rbac, dice_workflow, acute_change_screen, pii_scrubber.
models/	SQLAlchemy ORM: profile, conversation, daily_checkin, incident, behavioral_dossier, care_change_event, cross_patient_strategies, invite_code, profile_insights, response_feedback, safety_event, facility, staff, facility_patient_link, staff_patient_assignment, audit_log.
prompts/	Jinja2 templates: coach_system, checkin_system, learn_system, extraction_system, domain_knowledge, few_shot_examples.
config.py / db.py	Pydantic settings (lru_cache'd); async engine + pooled session factory.
alembic/, tests/	10 migrations; ~1,578 pytest-asyncio tests.

Key API surface

Method / Path	Notes
GET /health	DB connectivity gates HTTP 503; reports LLM availability (available / degraded / unknown) and rolling p50/p95 latency — informational only.
POST /api/profiles	Create profile (invite-code gated); returns an 8-char access code.
GET / PUT / DELETE /api/profiles/{code}	Lookup / update / cascading delete across 9 linked tables.
POST /api/coach/chat	Moment Coach — SSE streaming, the centerpiece endpoint (see §3.2).
POST /api/coach/acute-change-screen	Structured delirium/onset screening, run before a coach session for a sudden behavior change.
POST /api/checkin	Caregiver emotional check-in — SSE streaming.
GET /api/insights/{code}, /api/care-patterns/{code}, /api/strategies/{code}	Behavioral insights, episode-cycle risk, cross-patient strategy recommendations.
/api/facilities/*	Separate B2B staff-portal surface — JWT + PIN auth.

3.1 Streaming design (coach.py)

- FastAPI StreamingResponse with manually formatted SSE frames (data: {json}\n\n); headers set Cache-Control: no-cache and X-Accel-Buffering: no so the Railway/nginx proxy never buffers a chunk.
- First event is {"session_id": ...}; text arrives as {"text": chunk}; a {"replace": ...} event swaps an already-streamed partial response (used on mid-stream LLM failure and on failed post-stream validation); stream ends with [DONE].
- **Latency:** asyncio.gather() runs the RAG vector lookup concurrently with DB context reads (profile, dossier, incidents, cross-patient strategies) — different resources, so the caregiver waits max(rag, db) instead of the sum before the first token.
- **Background persistence:** assistant-message save, tag generation, incident extraction, and insights/dossier recompute are detached via asyncio.create_task, with strong references held in a module-level set (asyncio only weak-refs a bare task, so GC could otherwise cancel a half-finished write). A drain function gives in-flight writes up to 10s at shutdown.
- **B2C vs. B2B branching:** access_code (consumer) vs. profile_id (facility/staff, authorization-checked via staff_can_access_profile); audit attribution is derived only from the verified JWT, never the request body.
- **Locale-aware emergency numbers:** distinguishes the UI's locale_code from a broader emergency_locale geography map, so an English-UI caregiver physically in Paris is told 15/112, not 911.

3.2 Safety pipeline — the core safety-critical subsystem

Two layers, run in this order on every Moment Coach and Check-In message:

- **Deterministic regex gate** (safety_gate.py) — hand-curated patterns across 4 categories (life threat, self-harm, caregiver-harm-risk, elder abuse/neglect), checked in multiple languages regardless of the declared UI locale. Authoritative when it fires — bypasses the LLM entirely.

- **Heuristic fallback classifier** (`safety_classifier.py`) — runs only if the gate didn't fire. Explicitly documented as *not* a trained ML model; uses fuzzy character-similarity matching (`difflib.SequenceMatcher`, threshold 0.72) against curated concept phrases.

Real production bug, fixed

“He keeps wandering at night and I don't know what to do” — an ordinary Moment Coach message — was misclassified as suicidal ideation. Root cause: the character-level matcher scored “what to do” at ~0.76 similarity to “want to die” purely from shared letters, with no check that the two phrases shared any actual words. **Fix:** added a word-level content-word gate ahead of the character-level score — every non-stopword word in the concept phrase must first find a plausible match among the message's words before the character-ratio score is even computed. This preserved typo tolerance (“hart attak” → “heart attack”) while eliminating the false positive. Verified against the red-team suite (0 false positives, no sensitivity loss); the triggering message was added permanently to the regression dataset.

Output guardrail (`response_guard.py`)

After the LLM finishes streaming, `validate_response_quality()` checks language/script correctness (e.g. catches the model transliterating Hindi into Latin letters) and scans for disrespectful language (a blocklist plus negation-aware directive patterns like “control him”). On failure the LLM is asked to repair the response; if repair also fails, a static, localized fallback message is shown instead of raw garbage or silence.

3.3 LLM provider abstraction

- LLMProvider ABC with `stream_completion()` / `completion()`; concrete implementations for OpenAI (default, gpt-4o-mini) and Anthropic (claude-sonnet-4), switchable via env var.
- FallbackLLMProvider wraps primary + secondary. For streaming it can only fail over *before* the first chunk reaches the client (bytes already sent can't be retracted) — a mid-stream failure re-raises for the SSE layer's own fallback handling. Non-streaming calls retry fully on the secondary.
- Embeddings: OpenAI text-embedding-3-large (3072-dim). TTS: OpenAI gpt-4o-mini-tts, with automatic client-side fallback to the browser/device's own speech synthesis if unavailable.

3.4 Data storage & privacy

- **No server-side PII by design:** the patient's name lives only in the browser's localStorage — the server never receives it. Clinical fields (disease stage, behavioral patterns, calming strategies, safety concerns) are keyed by a **keyed HMAC-SHA256** hash of the access code — deterministic and therefore searchable, but not offline-brute-forceable without the server's HMAC secret.
- **Encryption at rest:** AES-256-GCM on all conversation content, profile fields, and feedback (format: "ENC:" + base64(nonce || ciphertext || tag)). The cipher object is lru_cache'd (building it runs a real key schedule, and dossier/insights recompute encrypts/decrypts hundreds of fields per pass). Decrypting a non-“ENC:” value logs a warning rather than erroring, to surface un-migrated plaintext rows during a safe rolling migration.
- **Cascading deletion:** `DELETE /api/profiles/{code}` erases the profile plus every linked record across 9 tables. No authorization beyond possessing the access code itself — documented as a deliberate scope limit, not an oversight.

3.5 B2B facility auth & authorization

- bcrypt for staff PIN and facility admin password hashing.

- HS256 JWTs with a **fail-closed** minimum secret length (32 chars) — refuses to sign or verify tokens below that length rather than silently accepting a forgeable secret.
- `staff_can_access_profile()` — per-request authorization so a facility-A staff JWT cannot read/write a facility-B patient (anti-IDOR).

■ Cross-tenant IDOR — found and fixed, confirmed exploitable in production

GET /api/facility/residents/{profile_id}/behavioral-card checked staff-role callers against their assignment, but admin/owner callers had no object-level authorization check at all — any authenticated facility admin/owner could pull another facility's (or an unrelated B2C profile's) behavioral card. Confirmed live against Railway before the fix. Resolved by routing admin/owner callers through the same `staff_can_access_profile()` helper, with new regression tests for same-facility (200), cross-facility (403), and unassigned-staff (403) access.

3.6 Rate limiting

In-process, per-client-IP sliding window (a deque of monotonic timestamps), applied via a FastAPI dependency on LLM-backed endpoints. Explicitly documented as **not distributed** — per-worker only, no cross-replica coordination — with a bounded per-request eviction sweep so memory doesn't grow unbounded with distinct client IPs.

3.7 Caching — what's real vs. observability-only

Mechanism	What it actually does
Anthropic prompt cache (anthropic_provider.py)	Real: wraps the ~6k-token system prompt with <code>cache_control</code> : ephemeral, opting into Anthropic's server-side prompt cache.
OpenAI cached-token reporting (openai_provider.py)	Passive: OpenAI auto-caches long stable-prefix prompts on its own; the code only reads back <code>usage.cached_tokens</code> , it doesn't enable anything.
token_usage.py rolling window	NOT a cache — a 200-sample in-process aggregator computing <code>avg_cached_tokens / cache_hit_rate</code> for /health. Pure observability on top of the providers above.
<code>config.get_settings()</code> — <code>@lru_cache(maxsize=1)</code>	Real: avoids re-parsing .env on every request. Gotcha: process-lifetime — a .env change needs a restart.
<code>crypto._get_cipher()</code> — <code>@lru_cache(maxsize=1)</code>	Real: caches the constructed AESGCM object, since building it runs a key schedule reused across hundreds of encrypt/decrypt calls per dossier pass.
speech.py TTS response headers	Real HTTP caching: <code>Cache-Control: private, max-age=3600</code> — synthesis is deterministic for a given text, so the browser can skip re-fetching it.
SSE endpoints (coach.py, checkin.py)	Deliberately uncached: <code>Cache-Control: no-cache</code> — correct, since these are live streams, not cacheable content.
DB layer (db.py)	Connection pooling only (<code>pool_size=20</code> , <code>pool_recycle=1800</code> , <code>pool_pre_ping</code>) — no query-result caching; dossier/insights recompute runs fresh every time.
RAG pipeline	No caching found — the vector store is persistence, not a cache layer.
Frontend / mobile localStorage & SecureStore	Identity/preference persistence (access code, patient name, theme), not a data cache — no TTL, no invalidation, doesn't avoid refetching server data.

4 Frontend Architecture (frontend/src)

Structure

- `app/[locale]/` — Next.js App Router pages under a locale segment: `home`, `coach`, `check-in`, `learn`, `incidents`, `journey/*`, `facility/*` (B2B), `login`, `impact`, `profile/*`.
- `components/ui/` — generic design-system components (`Button`, `Card`, `OfflineBanner`, `SpeakButton`, `MicButton`, `LocaleSwitcher`, `SafetyDisclosure`, `MedicalDisclaimer`, etc.), most with colocated `.test.tsx` files.
- `features/` — feature-scoped logic + UI: `checkin/CheckInScreen.tsx`, `coach/` (`CoachInput`, `CoachResponseRenderer`, `parseResponse.ts`, `useStreamingChat.ts`), `home/`, `incidents/`, `learn/`, `profile/` (`wizard`), `welcome/`.
- `lib/` — `api.ts`, `facility-api.ts`, `storage.ts` (`localStorage` wrappers), `locale.ts`, `theme.ts`. `hooks/` — `useNetworkStatus`, `useSpeechSynthesis`, `useSpeechRecognition`, `useRequireRole`. `context/` — `ProfileContext`, `FacilityContext` (plain React Context, no external state library).

4.1 Moment Coach rendering & the markdown allowlist

`parseResponse.ts` splits raw LLM text into the four named sections via machine markers, and `CoachResponseRenderer.tsx` renders each into its own visually distinct card. Both it and `CheckInScreen.tsx` render LLM output through `react-markdown` with an explicit **allowedElements allowlist** (`p`, `ul`, `ol`, `li`, `strong`, `em`, `h2/h3`, `br`, `blockquote`, `code`, `pre`) — deliberately excluding `img` and `a`, with `unwrapDisallowed` (strip disallowed tags but keep their text, so guidance is never silently dropped) and `skipHtml`. This is **defense-in-depth**: even though the system prompt constrains the model's output shape, the renderer never trusts it outright.

4.2 i18n / RTL

- `next-intl` with locale-prefixed routing; `SUPPORTED_LOCALES/DEFAULT_LOCALE` centralized in `lib/locale.ts`.
- Translation source of truth lives at repo-root `locales/`, synced into both frontend and mobile copies.
- Components consistently use **logical CSS properties** (`ps-5`, `border-s-2`, `start-4/end-4` instead of `pl-/border-l-/left-/right-`) so a future RTL language works without per-component rewrites — even though only LTR languages ship today.
- Deliberately only 3 languages ship (EN/ES/HI): the language registry, the `locales/` files, and both clients' `SUPPORTED_LOCALES` are kept in lockstep by a test, so nothing is advertised that the product cannot actually render.

4.3 Offline handling (`useNetworkStatus.ts`)

- Tracks `navigator.onLine` plus online/offline window events; starts at `null` (no navigator during SSR) and is treated as “assume online” everywhere, so there's never a false offline banner on first paint.
- Explicitly documented as **best-effort, link-layer only** — true even on a router with no real internet — and deliberately does not probe the backend.
- Detection-only: no request queueing, no background sync, no offline read caching (a deliberate, documented scope boundary, not an oversight).
- Consumed in `CheckInScreen.tsx` to pick distinct error copy: offline vs. generic API error vs. connection failure.

4.4 Speech synthesis (useSpeechSynthesis.ts)

- **Dual-path playback:** neural TTS (OpenAI, via an `<audio>` element) preferred, with automatic fallback to the browser's native `speechSynthesis` on any failure.
 - **Shared module-level state** via `useSyncExternalStore` so multiple `SpeakButton` instances (one per coach section, plus a page-level auto-speak effect) don't overlap or duplicate playback.
 - **Fixed bug — double-speak:** clearing `<audio>.src` itself fired the element's own error event, which was wired to trigger the browser-fallback path — every neural response played twice. Fixed by detaching handlers before clearing `src`.
 - **Fixed bug — auto-speak silently never playing:** per-instance `unmount` called the shared `stopAll()`, assuming instances only unmount together on navigation; in reality individual `SpeakButtons` mount/unmount one at a time as Moment Coach's sections stream in, silently invalidating an unrelated in-flight synthesis request. Fixed by tracking mounted-instance count and only stopping on the last one.
 - **Latency fix:** auto-speak now speaks the “Right Now” section as soon as it finishes streaming, instead of waiting ~20–30s for the full four-section reply before synthesizing anything.
 - Chrome-specific workaround for its silent ~15-second `speechSynthesis` cutoff (periodic keep-alive `resume()` calls).
-

5 Mobile App (mobile/)

Expo SDK 55, React Native, file-based routing via `expo-router` (“the file tree IS the navigation”). Own parallel — not literally shared — implementations of API/network/markdown logic that mirror the web client's conceptual design (same SSE contract, same `useNetworkStatus` pattern, same markdown-allowlist philosophy). Translations are the one genuinely shared source of truth.

Android hardening (app.json / expo-build-properties)

- `usesCleartextTraffic: false` — forces TLS to the backend.
- ProGuard + code minification + resource shrinking enabled for release builds.
- `allowBackup: false` — prevents Android's automatic app-data backup from silently copying locally-stored patient name / access code data to the user's Google account.
- `SYSTEM_ALERT_WINDOW` explicitly added to `blockedPermissions` — denies an overlay permission some libraries request by default.
- Speech/mic permission strings scoped tightly to the actual caregiving use case on both iOS (`NSSpeechRecognitionUsageDescription`, `NSMicrophoneUsageDescription`) and Android (`RECORD_AUDIO`, `MODIFY_AUDIO_SETTINGS`, `FOREGROUND_SERVICE*`).

OTA updates: `expo-updates` + EAS Update lets JS/UI fixes reach installed devices on next launch without a full APK rebuild; native-level changes still require one. **Distribution:** Android-only today by business decision (a paid Apple Developer account is the only blocker for iOS — the `eas.json` iOS config already exists).

6 Testing

Suite	Tooling	Current scale
Backend	pytest-asyncio	1,578 passing — covers safety gate/classifier/red-team, crypto, facility auth/IDOR, incident extraction, insights, RAG, rate limiting, response guard, PII scrubbing, fallback provider.
Frontend	Vitest + React Testing Library	256 passing, 1 intentionally skipped, 33 files.
Mobile	Jest + @testing-library/react-native	28 passing, 6 suites — explicitly flagged as thin coverage; native voice/OTA paths largely unverified by automated tests.

7 Notable Engineering Decisions

- **Safety-classifier false-positive fix** — a genuine precision/recall tradeoff solved by layering a word-level gate ahead of an existing character-level matcher, rather than replacing it (which would have broken typo tolerance), verified against a regression suite with zero sensitivity loss.
- **Cross-tenant IDOR fix on the facility behavioral-card endpoint** — a missing object-level authorization check on a privileged role, confirmed exploitable in production, fixed by reusing an existing authorization helper and backed by new regression tests for all three access scenarios.
- **RAG silently going dark after the infra migration** — after moving to Railway, the retrieval service was still pointed at a stale/default DB config and failed silently by design (fail-open, so a crisis response is never blocked on RAG being up) — meaning a real regression went unnoticed for a while purely because failures were invisible by design. A good resilience-vs-observability tradeoff to discuss.
- **Infra migration off a collaborator-hosted backend** — evaluated Oracle Cloud free tier (blocked on capacity) and Fly.io/Cloudflare Workers before landing on Vercel + Railway for cost and speed.
- **TTS double-speak bug** — a subtle DOM-event-ordering bug (clearing an audio element's src fires its own error event) causing every neural-voice response to play twice.
- **Deliberate 3-language scope discipline** — choosing an honest, smaller, fully-supported surface over an impressive-sounding but partially-unsupported one.
- **Honest, unresolved-risk documentation as a practice** — TODO(CLINICAL-LEGAL-REVIEW) markers on mandatory-reporting gaps, a red-team dataset whose reviewed_status field must never be silently flipped, and a running, dated, commit-stamped audit log (docs/AUDIT.md) that states exactly what was verified and what wasn't for every entry.